

Data Structures and Algorithms

A practical study reference with foundational concepts, examples, and common review questions.

1. Complexity

Time complexity describes how operation counts grow with input size n ; space complexity describes memory. Big O is an asymptotic upper bound, Omega a lower bound, and Theta a tight bound. Common growth rates include $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, and $O(2^n)$.

Analyze dominant terms and state assumptions. Best, average, and worst cases may differ. Dynamic-array append is amortized $O(1)$, although a resize can take $O(n)$. Measure real performance when constant factors and memory matter.

2. Linear, Hash, and Tree Structures

Arrays provide $O(1)$ indexed access; middle insertion usually shifts $O(n)$ values. Linked-list insertion is $O(1)$ when the node is known, but finding a position is $O(n)$. Stacks are LIFO; queues are FIFO; deques support both ends.

Hash-table lookup is expected $O(1)$ with suitable hashing and load factor, but worst-case $O(n)$. Balanced search trees support ordered operations in $O(\log n)$. A heap gives $O(1)$ access to the highest-priority item and $O(\log n)$ insertion/removal.

3. Searching and Sorting

Linear search takes $O(n)$. Binary search takes $O(\log n)$ on sorted random-access data; maintain a precise interval invariant. Merge sort is stable $O(n \log n)$ and uses extra memory; heapsort is $O(n \log n)$; quicksort is typically $O(n \log n)$ but can be $O(n^2)$; insertion sort suits small or nearly sorted input.

Stability preserves the input order of equal-key items. In-place algorithms use limited extra storage. Choose based on constraints and required guarantees, not only the headline complexity.

4. Graphs and Problem Solving

Graphs contain vertices and edges. Adjacency lists use $O(V+E)$ space; matrices use $O(V^2)$. BFS finds shortest paths in unweighted graphs in $O(V+E)$. DFS helps find components and cycles. Dijkstra requires nonnegative edge weights; Bellman-Ford can handle negative edges.

Dynamic programming stores overlapping subproblem results. Greedy algorithms need a proof that local choices produce a global solution. Clarify constraints, edge cases, invariants, data structures, and complexity before coding; test empty, duplicate, and boundary inputs.